

Анкета кандидата — Специалист по интеграции ИИ-агентов (IntellectDialog)

Время начала заполнения: 10:30

Вопрос 1: API — конкретный проект

Поле	Ответ
Название API	Kommo (amoCRM) REST API v4
Base URL	https://{subdomain}.kommo.com/api/v4/
Тип авторизации	Bearer (long-lived token), также поддерживает OAuth 2.0 для интеграций
Что конкретно делали	Разработал MCP-сервер (Model Context Protocol) с Telegram-ботом, который через AI (GPT с function calling) управляет CRM: полный CRUD сделок/контактов/компаний, аналитика воронок, массовые операции, настройка воронок по шаблонам, управление кастомными полями, зачистка аккаунта. Данные синхронизируются в PostgreSQL для тяжёлой аналитики.
Одна проблема и как решили	API Kommo не позволяет удалить воронку (pipeline), если в ней есть сделки — возвращает 400. При этом в документации это не описано явно. Решил так: перед удалением воронки бот получает все сделки в ней, перемещает их в главную воронку со статусом 143 (закрыта, не реализована), и только после этого удаляет воронку. Также столкнулся с тем, что системные поля (tracking_data — utm_source, gclid и т.д.) нельзя удалить через API, хотя DELETE-запрос возвращает 200 — пришлось фильтровать их по типу и коду перед попыткой удаления.

Вопрос 2: Напишите запрос

```
curl -X POST "https://api.example.com/v1/mobile?USER_ID=100&USER_HASH=abc123" \
-H "Content-Type: application/json" \
-d '{"Method": "getUser", "Parameters": {"Phone": "79001234567"}}'
```

Вопрос 3: Найдите ошибки

Ошибка 1: Content-Type указан как text/html, а API принимает JSON. Должно быть application/json.

Ошибка 2: В result данные извлекаются из response.data, но API возвращает результат в поле Parameters (не data). Правильный путь: response.Parameters → lessons.

Вопрос 4: Прочитайте лог

HTTP-статус 200, но Error: 1010 — это нормально? Да, это распространённая практика. Многие API (особенно legacy и CRM-системы) используют HTTP 200 для всех ответов, а бизнес-ошибки передают в теле ответа через собственные коды. Это позволяет отличить «сервер работает, но запрос некорректен» от «сервер упал». HTTP-статус говорит о транспорте, Error в теле — о бизнес-логике.

Что означает код 1010? Без документации точно не скажу, но по контексту (пустой Parameters) — скорее всего это ошибка авторизации, истёкший токен, или запрос с невалидными/отсутствующими обязательными параметрами. Возможно также — клиент не найден или метод недоступен для данного аккаунта.

Что бы сделал по-другому:

1. Не показывать пользователю общее «Произошла ошибка» — нужно парсить код ошибки и давать конкретное сообщение (например, «Не удалось найти ваш профиль» или «Сервис временно недоступен»).
2. Завести маппинг известных кодов ошибок → человекочитаемых сообщений.
3. Логировать Request_id для быстрой отладки с командой API.
4. Если ошибка повторяемая (например, 1010 = невалидный токен) — реализовать автоматический retry с обновлением токена, а не просить пользователя «попробовать позже».

Вопрос 5: Цепочка API-вызовов

Клиент: «Запиши меня на йогу завтра на 18:00»

Шаг 1: getClientByPhone

- Параметры: Phone (номер телефона клиента, берём из WhatsApp-профиля)
- Получаем: ClientID
- Зачем: идентифицировать клиента в системе
- Что может пойти не так: клиент не найден (нет в базе) → предложить зарегистрироваться или уточнить номер

Шаг 2: getSchedule

- Параметры: StartDate = завтрашняя дата 00:00, EndDate = завтрашняя дата 23:59
- Получаем: список занятий с ID, названием, временем, типом, свободными местами
- Зачем: найти занятие «Йога» в 18:00
- Что может пойти не так: нет занятий йогой завтра, нет занятий в 18:00, несколько занятий йогой в 18:00 (разные залы/тренеры) → показать варианты и уточнить

Шаг 3: Проверка свободных мест

- Из ответа getSchedule смотрим AvailableSlots для найденного занятия
- Что может пойти не так: AvailableSlots = 0 → сообщить что мест нет, предложить другое время или лист ожидания

Шаг 4: getClientAppointments

- Параметры: ClientID
- Получаем: список текущих записей клиента
- Зачем: проверить что клиент не записан уже на это занятие, и нет пересечений по времени
- Что может пойти не так: клиент уже записан → сообщить об этом

Шаг 5: Определяем тип занятия

- Из getSchedule берём Type: Group или Personal
- Если Group → используем SetGroupAppointment
- Если Personal → нужны getAvailableTrainers → getAvailableDates → getAvailableTimes → setAppointment

Шаг 6 (для группового): SetGroupAppointment

- Параметры: ClientID, AppointmentID (ID занятия из getSchedule)
- Получаем: подтверждение записи
- Что может пойти не так: место заняли пока шёл диалог (race condition), нет активного абонемента, ошибка API

Шаг 7 (опционально): getTickets

- Параметры: ClientID
- Зачем: проверить что у клиента есть действующий абонемент/членство, покрывающее это занятие
- Лучше вызвать до записи (между шагом 4 и 6), чтобы не получить ошибку при записи
- Что может пойти не так: абонемент истёк, не покрывает данный тип занятий

Вопрос 6: GPT и LLM

6А. Неверное утверждение

Номер: 2

Почему: function calling НЕ позволяет модели вызывать внешние API напрямую. Модель лишь генерирует JSON с именем функции и параметрами — это структурированный «запрос на вызов». Фактический вызов API выполняет наш код (оркестратор), который получает этот JSON, делает HTTP-запрос, и передаёт результат обратно модели. GPT не имеет доступа к сети и не может сама ничего вызвать.

6Б. Нестабильная работа LLM

В проекте KomtoMCP бот на gpt-4o при удалении кастомных полей сообщал пользователю «42 поля удалено, 19 не удалось» — но при повторном запросе списка все поля оставались на месте. Проблема: API возвращал HTTP 200 на DELETE системных полей (tracking_data), но фактически не удалял их. Модель видела в ответе `deleted: 42, failed: 19` и интерпретировала это как успех, хотя «deleted» включало и те поля, что API молча проигнорировал.

Что помогло: 1) Добавил фильтрацию системных полей (tracking_data, PHONE, EMAIL, WEB, ADDRESS, POSITION, IM) на уровне кода — они пропускаются и не считаются в deleted. 2) Возвращаю модели чёткий отчёт с разделением: `deleted_names`, `skipped_system`, `failed` — модель больше не может

неправильно интерпретировать результат. 3) Поднял модель до gpt-5-mini для лучшего следования инструкциям.

Вопрос 7: Два интента в одном сообщении

Какие функции должен вызвать роутер? Две: `schedule` (расписание на завтра после обеда) и `cancel` (отмена утренней йоги).

Если модель может вызвать только одну функцию за раз — как обработать? Решение: после выполнения первой функции и получения результата — вернуть управление роутеру с оставшейся частью запроса. Конкретно:

1. Роутер вызывает первую функцию (например, `cancel` — она критичнее).
2. После завершения отмены, оркестратор проверяет: остались ли необработанные интенты? Если да — снова вызывает роутер с оставшимся контекстом.
3. Альтернативный подход: роутер возвращает массив интентов с приоритетами, оркестратор выполняет их последовательно.

Описание функции `schedule` (чтобы не путать с `book`): «Возвращает расписание занятий на указанный период. Только просмотр — НЕ записывает клиента. Используй когда клиент хочет узнать какие занятия есть, посмотреть расписание, спрашивает что доступно.»

Вопрос 8: System prompt — сравнение

Какой вариант лучше для production? Вариант Б.

3 конкретные проблемы варианта А:

1. **Галлюцинации:** нет запрета выдумывать данные — бот будет придумывать названия занятий, тренеров и расписание из своих обучающих данных вместо вызова API.
2. **Нет управления состоянием:** без `params` бот не помнит на каком шаге диалога находится — при многошаговой записи (выбор даты → выбор занятия → подтверждение) будет терять контекст или перескакивать шаги.
3. **Нет граничных условий:** не описано что делать когда мест нет, когда несколько занятий с одним названием, когда клиент не указал дату — бот будет принимать решения сам, непредсказуемо (например, подставит сегодняшнюю дату без спроса).

Зачем строка «Продолжай строго с объектом `params`. Не создавай новый»? LLM склонна «обнулять» состояние между вызовами — вместо продолжения существующего объекта `params` она может создать новый с дефолтными значениями, потеряв всё что клиент уже выбрал (дату, занятие, тренера). Эта инструкция заставляет модель работать с переданным состоянием как с мутабельным объектом, а не пересоздавать его.

Вопрос 9: Галлюцинации и достоверность

Почему бот мог так ответить? GPT обучена на огромном корпусе текстов, включая описания фитнес-клубов, расписания, имена тренеров. Когда модель не вызвала инструмент `getSchedule`, она

сгенерировала правдоподобный ответ из своих обучающих данных — «Hot Yoga», «тренер Алексей», «19:30» звучат реалистично. Модель оптимизирована быть полезной и давать ответ, даже если данных нет — это и есть галлюцинация.

3 приёма чтобы предотвратить:

1. **В промпте:** явный запрет — «НИКОГДА не называй занятия, тренеров, время или цены, если ты не получил эти данные из инструмента. Если данных нет — скажи что нужно проверить и вызови инструмент.»
2. **В архитектуре:** сделать вызов `getSchedule` обязательным шагом в пайплайне до ответа пользователю — не давать агенту возможность ответить про расписание без предварительного вызова API. Например, через `checkConditions`: если ответ содержит время/название занятия, но `getSchedule` не вызывался → блокировать ответ.
3. **В коде:** валидация ответа — перед отправкой пользователю проверять что упомянутые ID занятий, имена тренеров и время реально существуют в последнем ответе API. Если нет — перезапросить у модели.

«Не выдумывай» vs «не давать данные без инструмента» — что надёжнее? Второй подход надёжнее. Инструкция «не выдумывай» — это *soft constraint*, модель может её нарушить (особенно при длинных диалогах, когда инструкция «размывается»). А архитектурное ограничение — *hard constraint*: если данные физически не попадают в контекст модели без вызова инструмента, она не сможет их выдумать в правильном формате. Промпт — это пожелание, архитектура — это гарантия.

Вопрос 10: Дебаг нестабильной работы

Где искать причину (3+ гипотезы):

1. **Общий контекст между сессиями:** если *conversation history* не изолирована по пользователям — ID занятий из предыдущего разговора с другим клиентом попадают в контекст текущего. Это прямо подтверждается логами («иногда бот использует ID из предыдущего разговора с другим клиентом»).
2. **Кэширование ответа `getSchedule`:** если результат `getSchedule` кэшируется и не обновляется между сессиями — бот может использовать устаревшие ID.
3. **Промпт не привязывает ID к конкретному выбору:** модель видит список занятий, клиент говорит «второе» или «йогу в 18:00», и модель неправильно матчит текстовый выбор с ID из списка.
4. **Race condition:** между показом расписания и записью прошло время, расписание обновилось, ID сместились.

Самая вероятная гипотеза: Гипотеза 1 — общий/протекающий контекст. Логи прямо говорят что бот берёт ID из разговора с другим клиентом. Это значит что *conversation history* либо глобальная, либо неправильно очищается между сессиями. 70/30 соотношение объясняется тем, что в большинстве случаев ID совпадают (популярные занятия), но когда предыдущий клиент спрашивал про другое — ID «протекает».

Как исправить:

- 1. Изолировать conversation history строго по user_id/chat_id — каждый клиент видит только свой контекст.
- 2. Очищать контекст при начале нового сценария записи.
- 3. Добавить валидацию: перед вызовом SetGroupAppointment проверять что AppointmentID существует в последнем ответе getSchedule для ЭТОГО пользователя.
- 4. В промпте явно указать: «Используй ТОЛЬКО ID занятий из последнего вызова getSchedule. Никогда не используй ID из предыдущих разговоров.»

Вопрос 11: RAG — когда нужен и когда нет

Источник данных	Способ	Почему
Расписание занятий	API	Обновляется ежедневно, нужны актуальные данные в реальном времени. Кэш в промпте или RAG устареет за часы. Бот должен вызывать getSchedule при каждом запросе.
Прайс-лист	System prompt	10 позиций — это ~200-300 токенов, легко помещается в промпт. Обновляется раз в полгода — нет смысла усложнять архитектуру. При изменении — обновить промпт.
FAQ (50 вопросов)	RAG	50 Q&A — это слишком много для промпта (~5000+ токенов, засорит контекст). Но и API не нужен — данные статичные. RAG найдёт 2-3 релевантных ответа по запросу и подставит только их.
Информация о тренерах	RAG	15 тренеров × 2-3 абзаца = ~3000-4500 токенов. В промпт не влезет без потери качества. RAG подтянет инфу только о тренере, про которого спрашивают. Обновляется редко — не нужен API.

Вопрос 12: Конструкторы и визуальные workflow

12А

- ☒ n8n
- ☐ Make (Integromat)
- ☐ Zapier
- ☐ Flowise
- ☒ Langchain / LangGraph
- ☐ Voiceflow / Botpress

12Б

Поле	Ответ
Какая платформа	n8n

Поле	Ответ
Для какой задачи	Интеграция Kommo CRM с AI-ботом через MCP-сервер: webhook принимает событие из Kommo → обогащает данные → вызывает MCP-инструменты → отправляет результат в Telegram
Сколько нод/шагов	~15-20 нод в самом сложном workflow (webhook → HTTP request → IF → switch по типу события → несколько веток обработки → ответ)
Что бесило больше всего	Отладка ошибок в цепочке: когда нода падает на 7-м шаге, приходится прокликивать каждую предыдущую ноду чтобы понять какие данные пришли. Нет нормального «дебаг-режима» с breakpoints.

12B

1. Зачем filter → save_variables перед agent? Два промежуточных шага нужны чтобы: (а) filter убирает нерелевантные данные — из 100 занятий оставляет только групповые с местами на нужную дату, иначе агент получит огромный JSON и может запутаться или превысить контекст; (б) save_variables форматирует сырой JSON в читаемый текст/структуру для агента — вместо `{"ID": 1, "Service": {"Title": "Йога"}, "StartDate": "2026-02-11T09:00:00", "AvailableSlots": 3}` агент получит «Йога — 09:00 (3 места)». Это экономит токены и снижает вероятность ошибки.

2. Зачем runScenario? DRY-принцип: одна и та же цепочка (api → filter → save_variables) нужна и в ветке «расписание» и в ветке «записаться». Без подсценария пришлось бы дублировать 3 ноды. При изменении логики фильтрации (например, добавить фильтр по абонементу) — правишь в одном месте, а не в двух. Также упрощает тестирование — можно протестировать подсценарий изолированно.

3. Что проверяет checkConditions после SetAppointment? Проверяет поле Error в ответе API. Ветки:

- Error = 0 (успех)** → getResult с сообщением «Вы записаны на [занятие] в [время]» + кнопка «Отменить запись»
- Error ≠ 0 (ошибка)** → дальше может быть ещё одно ветвление по коду ошибки: нет мест → предложить другое время; нет абонемента → предложить купить; клиент уже записан → сообщить об этом; другая ошибка → «Не удалось записать, попробуйте позже» + логирование

Вопрос 13: JSON и код

Какой вариант надёжнее? Вариант Б.

Что делает regex `/^[1-9]\d{10,14}$/`? Проверяет что строка — это валидный номер телефона: начинается с цифры 1-9 (не с нуля), далее от 10 до 14 цифр. Итого 11-15 цифр — покрывает форматы от российских (79001234567, 11 цифр) до международных (до 15 цифр по стандарту E.164).

Что делает `.replace(/D/g, '')`? Удаляет все нецифровые символы из строки. `\D` — любой символ кроме цифры, `g` — глобально (все вхождения). Превращает `+7 (900) 123-45-67` в `79001234567`.

Пример сообщения где А сохранит мусор, а Б — корректный номер: Пользователь пишет: `Мой номер +7 (900) 123-45-67, запишите меня`

- Вариант А сохранит: **Мой номер +7 (900) 123-45-67, запишите меня** — весь текст сообщения как «телефон»
- Вариант Б: очистит от нецифровых символов → **79001234567**, проверит regex → валидный номер, сохранит **79001234567**

Вопрос 14: API не совпадает с документацией

Что делаю первым делом? Составляю таблицу из 15 методов с тремя колонками: ☒ работает (8), ☐ работает с отличиями (3), ☒ не существует (4). Для каждого проблемного метода фиксирую конкретную ошибку: какой запрос отправил, что ожидал, что получил.

Как организую информацию? Таблица в Google Sheets / Notion, расшаренная с командой и клиентом:

Метод	Статус	Проблема	Что нужно от клиента
getUser	☒	—	—
getSchedule	☐	Ожидает формат DD.MM.YYYY, в доке YYYY-MM-DD	Подтвердить формат
deleteUser	☒ 404	Метод не существует	Актуальный endpoint

Что напишу клиенту: «Протестировал все 15 методов из документации. 8 работают, 3 работают с отличиями от документации (подробности в таблице — нужны уточнения по формату), 4 метода возвращают 404 — возможно переименованы или ещё не реализованы. Пришлите актуальную документацию или дайте контакт разработчика API для уточнений. Параллельно начинаю строить бота на 8 рабочих методах.»

Ждать ли починки всех 15? Нет. Начинаю строить бота на 8 рабочих методах — это уже покрывает основной функционал. 3 метода с отличиями скорее всего починю сам (поменять формат даты, добавить параметр). 4 отсутствующих — эскалирую клиенту, но не блокирую работу. К дедлайну будет рабочий бот с 8-11 методами, оставшиеся подключу когда клиент предоставит рабочие endpoints.

Вопрос 15: Приоритизация задач

Порядок: Б → В → А

- Задача Б (фитнес-клуб, баг)** — ПЕРВАЯ. 500 клиентов в день не могут записаться на персональные тренировки, клиенты жалуются. Это production-инцидент с прямыми финансовыми потерями для существующего клиента. Каждый час простоя = потерянные записи и репутация. Скорее всего сломалось из-за изменения API после обновления CRM — нужно найти что поменялось и адаптировать.
- Задача В (QR-код пропуска)** — ВТОРАЯ. Полдня работы, один API-метод, быстрый результат. Пока жду ответы от клиента А по документации — закрою эту задачу. Клиент получит фичу, я разблокирую его запрос.

3. **Задача А (автосалон, новое подключение)** — ТРЕТЬЯ. 3-5 дней работы, документация частично устаревшая — будут вопросы к клиенту, ожидание ответов. Нет срочности — это новый клиент, ещё не в production. Начну после закрытия Б и В.

Вопрос 16: Кодовая задача

```
import json
from datetime import datetime

response = {
    "Error": 0,
    "Parameters": [
        {"ID": 1, "Type": "Group", "Service": {"Title": "Йога"}, "StartDate":
"2026-02-11T09:00:00", "AvailableSlots": 3},
        {"ID": 2, "Type": "Personal", "Service": {"Title": "Бокс"}, "StartDate":
"2026-02-11T10:00:00", "AvailableSlots": 0},
        {"ID": 3, "Type": "Group", "Service": {"Title": "Пилатес"}, "StartDate":
"2026-02-11T11:00:00", "AvailableSlots": 5},
        {"ID": 4, "Type": "Group", "Service": {"Title": "Йога"}, "StartDate":
"2026-02-11T18:00:00", "AvailableSlots": 1},
        {"ID": 5, "Type": "Group", "Service": {"Title": "Stretching"},
"StartDate": "2026-02-12T09:00:00", "AvailableSlots": 8},
    ]
}

target_date = '2026-02-11'

filtered = [
    item for item in response['Parameters']
    if item['Type'] == 'Group'
    and item['AvailableSlots'] > 0
    and item['StartDate'].startswith(target_date)
]

for item in filtered:
    time = datetime.fromisoformat(item['StartDate']).strftime('%H:%M')
    title = item['Service']['Title']
    slots = item['AvailableSlots']
    word = 'место' if slots == 1 else 'места' if 2 <= slots <= 4 else 'мест'
    print(f'{title} – {time} ({slots} {word})')
```

Вывод:

```
Йога – 09:00 (3 места)
Пилатес – 11:00 (5 мест)
Йога – 18:00 (1 место)
```

Вопрос 17: Архитектура

Запись выполняет **наш код**, не GPT. GPT не имеет доступа к сети и не может вызывать API напрямую. Когда клиент пишет «запиши меня на йогу», GPT анализирует запрос и генерирует структурированный JSON через function calling — например, `{"name": "SetGroupAppointment", "arguments": {"ClientID": 123, "AppointmentID": 456}}`. Наш код-оркестратор получает этот JSON, выполняет реальный HTTP-запрос к API фитнес-клуба, получает ответ, и передаёт результат обратно GPT — которая уже формулирует человекочитаемый ответ клиенту.

Вопрос 18: Самооценка

18А

Навык	Оценка	Комментарий
REST API (тестирование, отладка)	5	Ежедневно работаю с Kommo API v4 — CRUD, пагинация, rate limits, OAuth, обход ограничений документации.
JSON (чтение, редактирование)	5	Работаю с вложенными JSON-структурами CRM-ответов (embedded entities, custom fields, nested links) каждый день.
Python (скрипты)	5	Основной язык — весь KommoMCP написан на Python (asyncio, aiohttp, PostgreSQL, 4000+ строк основного модуля).
GPT / LLM (промпты, function calling)	4	Реализовал итеративный tool calling с RAG-подбором инструментов, но опыт в основном с OpenAI API, меньше с fine-tuning и evaluation.
Визуальные конструкторы (n8n, Make)	3	Работал с n8n для интеграций, но не строил сложные production-боты в визуальных конструкторах — больше кодом.
RAG / базы знаний	4	Реализовал RAG-систему с keyword matching для динамического подбора инструментов из YAML-реестра, но не работал с векторными БД (Pinecone, Weaviate).
Отладка чужого кода и конфигов	5	Регулярно дебажу проблемы на стыке API + LLM + бот — от неправильных HTTP-статусов до галлюцинаций модели.

18Б

Не работал плотно с визуальными конструкторами ботов типа Voiceflow/Botpress и конкретно с платформой IntellectDialog. Учитывая опыт с n8n и понимание архитектуры нод/графов (описанной в вопросе 12B) — разберусь за 3-5 дней активной работы с платформой. Также не работал с Make/Zapier, но принцип тот же — 1-2 дня на освоение.

Время окончания заполнения: 11:15